

Liivv — Architecture Overview

Audience: Product, ops, and anyone who needs the shape of the app

App: BigCommerce Catalyst storefront (core/) + Liivv health and pharmacy extensions

Date: August 2026

Status: Current

Companion: [Deep dive](#) — data flows, auth, secrets, security checklist S1–S8, evidence paths

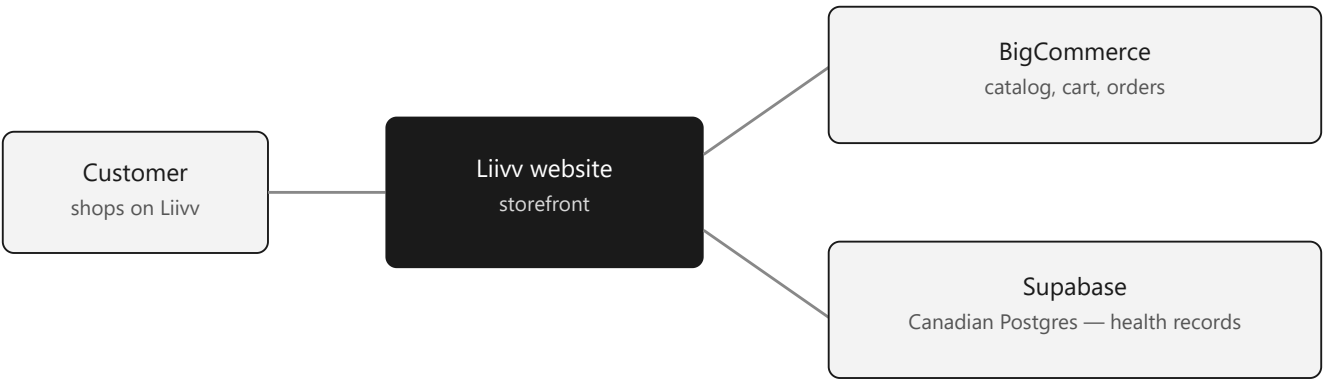
PDF: [Liivv-Architecture.pdf](#)

1. What Liivv is

Liivv is a health storefront. Shoppers see one site. Behind it, **commerce and health data use different systems of record**.

- **BigCommerce** runs the shop: catalog, cart, checkout, orders, customer login.
- **Supabase** holds health records in a **Canadian** Postgres project: profile, insurance, prescriptions, CarePack, care chat.
- **Stripe** takes card charges and subscriptions. Card numbers never sit in Liivv.

Orders are not stored in Supabase. Health records are not stored in BigCommerce.



How a visit splits: customer shops on Liivv, then shopping goes to BigCommerce and health records go to Supabase

Function	System	Boundary
Catalog, cart, checkout, order of record	BigCommerce	Commerce platform — not health records
Health profile, insurance, prescriptions, care chat	Supabase	Canadian Postgres — not the shop
Card charges and subscriptions	Stripe	Payment processor — PAN / CVC never stored

Commerce platforms are built to sell products. Health records need a dedicated store with Canadian residency and access control. Combining them would put PHI in a system that is not designed to hold it.

Design principle: browsers never receive service-role, admin, or payment secrets. **Next.js on Vercel is the trust boundary.**

2. What each system is for

BigCommerce — the only store engine

Liivv does not invent a second checkout or process orders itself.

Handles: product catalog and prices, cart and checkout, customer accounts used to sign in, the official order after payment, and most customer email (password reset, order mail).

Does not handle: health questionnaires, insurance, prescriptions, refills, CarePack requests, or care-team conversations.

Recurring orders: Stripe handles the repeating charge. When a renewal succeeds, Liivv still creates the order in BigCommerce. Stripe is the billing engine. BigCommerce remains the only order engine.

Supabase — health records only

A professionally run Postgres database (plus optional Storage) for records that do not belong in the commerce platform.

- **Holds:** health profile, insurance, prescriptions, refill/CarePack requests, care chat.
- **Does not:** take payments or create orders.
- Clients never talk to Supabase directly. Liivv’s server holds the service-role key.

This project’s Supabase region is **Canada (Central) / ca-central-1**.

Patients add prescriptions by **transfer from another pharmacy** or **doctor fax** — not by uploading photos. Staff work the queue in a BigCommerce embedded app (/bc-app).

Vercel — where the app runs

The storefront is **Next.js 16** (App Router), deployed on **Vercel**. That is the hosting platform: production and preview deploys, environment secrets, and the Node runtime that talks to BigCommerce, Supabase, and Stripe.

Vercel is not a system of record. Orders stay in BigCommerce. Health rows stay in Supabase. Vercel holds the running app and the secrets it needs.

Everything else, in one pass

Layer	Technology	Role
Storefront / hosting	Next.js 16 on Vercel	Only app that holds secrets; UI; server actions; webhooks
AI assistant	OpenAI (optional, feature-flagged)	Store assistant — off until Legal decides on a DPA
Drug reference	Health Canada DPD	Free public drug catalog — customer medication search

Health Canada DPD — medication lookup

The **Drug Product Database (DPD)** is Health Canada’s **free public API** for licensed drugs in Canada — open government data, no API key, and no paid contract. Liivv uses it so customers can search by brand name and pick the right product when they add a prescription, instead of typing a DIN from memory.

The browser never talks to Health Canada. Next.js proxies the search and rate-limits it. What the customer selects is saved on their **prescription in Supabase**. DPD is a lookup catalog only; it does not hold patient records.

Care conversations live in Supabase. An on-site assistant (Olivia) can help with products, orders, and account questions **when we turn it on**. It is not a clinician. Human care-team chat in /bc-app stays in Supabase either way.

3. What is health data vs shop data

PHI is protected health information: data that identifies a person and says something about their health.

Production intent is a **paid Canadian Supabase project**. HIPAA is a US statute; the Canadian frame is **PIPEDA** (and **PHIPA** in Ontario). Vendor extras help; staff access, retention, and keeping chat out of uncovered tools remain Liivv’s responsibility.

Data class	Examples	Storage
PII	Name, email, address, BC customer id	BigCommerce + Supabase profiles
PHI / PHI-adjacent	Health categories, insurance, Rx, CarePack, chat	Supabase
Payment card data	PAN / CVC	Never stored — Stripe Elements
Commerce	Orders, SKUs, prices	BigCommerce (+ Stripe for billing)

4. How a visit works

Shopping. The browser talks only to Next.js. Next.js reads the catalog and cart from BigCommerce GraphQL, then returns HTML. At checkout, Next.js creates a Stripe Payment Intent; the customer confirms with Stripe.js. After Stripe says the charge succeeded (webhook), Next.js creates the order in BigCommerce Admin REST and clears the cart. Recurring lines become Stripe subscriptions; later renewals still land as BigCommerce orders.

Health and pharmacy. A logged-in customer completes profile, health, and insurance on the storefront. Next.js writes those rows to Supabase and may sync name/phone back to BigCommerce. To add a prescription, they search **Health Canada DPD** (free public drug API) for the brand, pick the product, and Liivv saves that medication on their prescription in **Supabase**. Transfer / fax / refill / CarePack requests are also Supabase rows. Staff approve and update them from /bc-app.

Care chat. Customer messages are stored in Supabase. Staff join and reply from the same staff app. The optional OpenAI bot is **off** until Legal signs a DPA; if it were on, message text would go to OpenAI even when the reply is “talk to a pharmacist.”

5. Who can do what

Who	How they get in	What they see
Customer	BigCommerce login (Auth.js session)	Storefront, their own health forms, their own chat
Anonymous shopper	Signed cart cookie	Catalog and cart only
Staff	BigCommerce app session inside the control panel	Pharmacy queue and care chat in /bc-app

There is no shared staff dashboard password. /staff is not a product surface. /admin only redirects to the BigCommerce control panel.

6. Security posture (high level)

Engineering treats Next.js as the only place secrets live. A few controls that matter in conversation:

- **Row Level Security** is on in Supabase, with no browser-facing database key. The server uses the service role.
- Health data stays in **Canadian** Supabase, not in BigCommerce.
- Staff access is the **store's BC app session**, not a Liivv password.
- The **OpenAI assistant stays off** until a DPA is in place.
- Secrets live in Vercel and gitignored `.env.local` — nothing privileged is committed.
- Stripe and BigCommerce **webhooks are verified**.
- Medication search is a **server proxy** to Health Canada's free public DPD API, not an open relay. Chosen medications are stored on the prescription in Supabase.

The numbered pre-launch checklist (**S1–S8**), cookie names, secrets inventory, and code evidence paths live in the [deep dive](#).

7. Still with Legal and ops

1. Vendor **DPAs / BAAs** as required (Supabase, Stripe, Vercel, BigCommerce; OpenAI only if the bot is turned on).
 2. Logging policy: do not print chat bodies in request logs.
 3. Backup / RPO for Supabase Postgres.
 4. Optional hardening: network restriction to Vercel; separate Preview vs Production secrets.
-

For sequence diagrams, systems of record, auth cookies, the S1–S8 table, and file paths, see [Liivv-Architecture-Deep-Dive.md](#).